Atlantic Technological University Galway
Department of Computer Science and Applied Physics

A Privacy-Preserving Edge Gateway for Real-Time Audio Deepfake Detection

Confidential Inference on ARM TrustZone: the Full Project Journey

A narrative account of the design decisions, alternatives, risks, and points of difficulty encountered while building and validating the confidential-inference component of the thesis

Author: Waleed Asif
Programme: MSc Computing
Supervisor: Dr. Owen Foley
Institution: Atlantic Technological University Galway
Document date: September 2026

Prepared for supervisory review — this document tells the complete story of the confidential-inference proof-of-concept: what was built, why each method was chosen over the alternatives, where the work got stuck, and how each obstacle was resolved.

Contents

1 Introduction **3**
- 1.1 What this document is . . . 3
- 1.2 The thesis in one paragraph . . . 3
- 1.3 Why confidential inference, specifically . . . 3

2 Background **5**
- 2.1 The detection backbone: RawTFNet-16 . . . 5
- 2.2 ARM TrustZone and OP-TEE . . . 7

3 Design Decisions and Alternatives Considered **8**
- 3.1 Emulation platform: QEMU ARMv8 "virt", not a Raspberry Pi 4 emulator . . . 8
- 3.2 Development host: VirtualBox Ubuntu 22.04 LTS, not WSL2 . . . 9
- 3.3 Model deployment path: PyTorch → ONNX → C, not hand-ported C . . . 9
- 3.4 Apache TVM: used for independent verification, not for the deployed code . . . 10
- 3.5 Scope of the proof-of-concept . . . 10

4 System Architecture **11**

5 The Implementation Journey **13**
- 5.1 Phase 1 — Environment bring-up . . . 13
- 5.2 Phase 2 — Validating the TA/CA communication pattern . . . 13
- 5.3 Phase 3 — Exporting and compiling the model . . . 14
  - 5.3.1 Export to ONNX and numerical verification . . . 14
  - 5.3.2 The abandoned path: onnx2c . . . 14
  - 5.3.3 The adopted path: emx-onnx-cgen . . . 14
- 5.4 Phase 4 — The memory wall . . . 15
  - 5.4.1 The failure . . . 15
  - 5.4.2 Investigation timeline . . . 16
  - 5.4.3 The self-caught misdiagnosis . . . 17
- 5.5 Phase 5 — The fix: partitioning the model . . . 17
  - 5.5.1 Verifying the split is lossless . . . 18
- 5.6 Phase 6 — Any-audio pipeline and proof of secure-world execution . . . 18
  - 5.6.1 The any-audio pipeline . . . 18
  - 5.6.2 Proving *which* audio was processed, and that it happened in the secure world 18
- 5.7 Phase 7 — The domain-generalisation investigation . . . 19
  - 5.7.1 Why this is framed as a finding, not a failure . . . 21
- 5.8 Phase 8 — Live Asterisk call integration . . . 21
  - 5.8.1 Design: sliding window, not record-then-analyse . . . 22
  - 5.8.2 What this requires from Asterisk, and what was actually built for this PoC 22

6 Verification Summary **24**

7 Limitations and Threats to Validity **25**

8 Positioning Against the Literature Review **26**

9 Current Status and Next Steps **27**
9.1 Where the project stands 27
9.2 Immediate next steps 27

10 Summary **28**

Chapter 1

Introduction

1.1 What this document is

This document is a narrative account, not a formal thesis chapter. Its purpose is to walk through the confidential-inference proof-of-concept (PoC) from the beginning: what problem it solves, why each significant design decision was made the way it was, which alternatives were considered and rejected, and — most importantly for a supervisory review — exactly where the project got stuck, what the failure looked like, and how it was diagnosed and resolved. Numeric results, verification evidence, and known limitations are reported honestly, including one negative research finding that turned out to be a legitimate contribution in its own right rather than a bug to hide.

The account is organised chronologically, in the order the work actually happened, because the reasoning behind later decisions frequently depends on what was learned at an earlier stage.

1.2 The thesis in one paragraph

The overall thesis proposes a privacy-preserving edge gateway for real-time audio deepfake detection in residential telephony: a small device sitting between a phone line and the call itself, listening for synthetic-speech characteristics while the call is in progress, and doing so without exposing the caller's voice or the detection model to whatever else is running on that device. The detection backbone is a pretrained lightweight neural network, RawTFNet-16 (checkpoint `Best_RawTFNet_16.pth`). The confidentiality property is provided by ARM TrustZone, via the open-source OP-TEE trusted execution environment. The intended deployment target is a Raspberry Pi 4. No physical Pi 4 was available during this phase of the work, so the entire proof-of-concept described here was built and verified on an emulated ARMv8-A platform instead — a substitution that is itself one of the first design decisions this document justifies.

1.3 Why confidential inference, specifically

A plain spoof-detection classifier running as an ordinary process is not enough for the threat model this thesis targets. Two properties distinguish the intended system from a bare classifier:

- **Live, not post-hoc, detection.** A decision has to be available while the call is still connected, using a rolling audio buffer, rather than analysing a recording after the call has ended.
- **Confidential inference.** The input is a person's voice and the model represents proprietary work; the inference computation itself should be shielded from a compromised or malicious host operating system. This is what ARM TrustZone provides: a hardware-enforced isolated execution mode where code and memory can be marked secure-world-only, so that even a

fully compromised normal-world OS — kernel included — cannot read or tamper with what runs inside it.

No existing peer-reviewed work combines federated learning, differential privacy, and TrustZone-based confidential inference specifically for anti-spoofing (see Chapter 8). This proof-of-concept is the first concrete step toward closing that gap for the thesis: it establishes that the confidential-inference half of that combination is achievable, before the federated-learning and differential-privacy halves are layered on top in later work.

Chapter 2

Background

2.1 The detection backbone: RawTFNet-16

RawTFNet (Xiao, Dang and Das, *RawTFNet: A Lightweight CNN Architecture for Speech Anti-spoofing*, APSIPA ASC 2025) is a lightweight, end-to-end anti-spoofing network. It takes the raw audio waveform directly as input — there is no hand-crafted front end, no MFCCs, no spectrogram, no CQT. All feature extraction happens inside the network's own learnable Sinc layer. Each utterance is cropped or zero-padded to a fixed-length segment of approximately four seconds (64,600 samples at 16 kHz in this project's checkpoint), and that fixed length is structural: it cannot be shortened without the network's internal pooling operations collapsing to zero size (a failure encountered directly and discussed in Section 5.3.1).

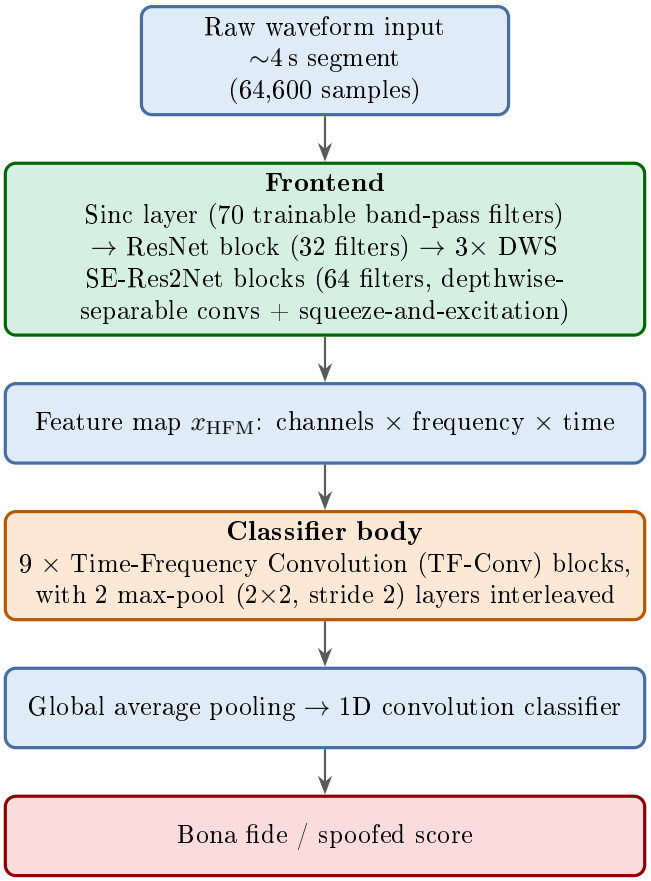


Figure 2.1: RawTFNet-16 architecture as trained and used unmodified as the detection backbone. Channel-width parameter $\tau = 16$: 0.07M parameters, 2.9G multiply–accumulate operations (MACs) per 4-second window. The green/orange split marks exactly where this project later cut the network in two — see Section 5.5.

Table 2.1 places RawTFNet-16 against the other anti-spoofing systems it was benchmarked against in the source paper. It is the smallest model in the comparison — an order of magnitude fewer parameters than RawNet2 — while reaching the lowest equal error rate (EER) on the ASVspoof 2021 LA evaluation set. That combination of footprint and accuracy is what makes it a realistic choice for confidential inference inside a memory-constrained TrustZone enclave, and is why it was selected as this project's detection backbone rather than a larger architecture such as AASIST or RawNet2.

Model	Params	MACs	2021 LA EER	2021 DF EER
RawNet2	17.60M	0.8G	9.50%	22.38%
AASIST	0.30M	8.9G	10.51%	21.07%
SE-Rawformer	0.37M	6.1G	4.98%	20.26%
RawTFNet-16	**0.07M**	**2.9G**	**4.50%**	18.53%
RawTFNet-32	0.17M	5.4G	5.05%	**16.82%**

Table 2.1: Single-system comparison on ASVspoof 2021 LA and DF (lower EER is better). Source: Xiao, Dang and Das, APSIPA ASC 2025.

Two properties of the published model directly shaped later engineering decisions in this project:

- **Duration sensitivity.** EER is worst on very short clips (0–2 s, over 10%), best around 2–4 s (matching the training segment length), and rises again for 4–8 s clips. This is why the

live-detection design (Section 5.8) uses a 4-second sliding window rather than a shorter one, even though a shorter window would reduce latency.

- **Ablations on the TF-Conv design.** Removing the frequency branch raises EER from 5.05% to 6.91%; removing the time branch raises it to 7.08%; removing the channel shuffle raises it to 5.74% at no change in model size. Both branches contribute meaningfully, which is one more reason the architecture was used exactly as published rather than simplified for easier embedding.

No changes were made to the network topology, layer widths, or channel-width parameter $\tau = 16$ anywhere in this project. Every adaptation described in this document is in *how the already-trained model is deployed* — exporting it, compiling it, and running it inside a trusted application — never in the network itself.

2.2 ARM TrustZone and OP-TEE

ARM TrustZone splits a single physical CPU into two isolated execution states: a *normal world* and a *secure world*, enforced in hardware rather than by the operating system. Code, memory, and peripherals can be marked secure-world-only, so that even a fully compromised normal-world operating system — kernel included — cannot read or tamper with what runs in the secure world.

OP-TEE is the open-source trusted OS that runs inside the secure world and exposes this split through a standard programming model with two halves:

- **Trusted Application (TA)** — code that executes inside the secure world. TAs are written in C, compiled against OP-TEE's TA development kit, cryptographically signed, and loaded on demand into a minimal, single-threaded environment with a small, fixed secure-memory heap.
- **Client Application (CA)** — an ordinary Linux program in the normal world. A CA opens a session with a TA and invokes commands through the GlobalPlatform TEE Client API (`libteec`).

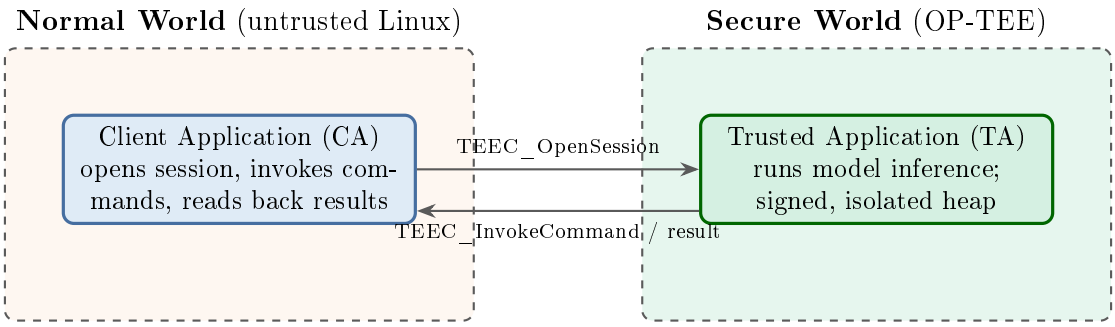


Figure 2.2: The TA/CA session pattern: open a session, invoke a command, read back a result. This is the exact mechanism that carries an audio window into the secure world and a spoof score back out. It was validated end-to-end using the official `hello_world` example before any model-specific code was written (Section 5.2).

Chapter 3

Design Decisions and Alternatives Considered

Every major decision in this project was a choice between at least two realistic options. This chapter records each decision, the alternative that was rejected, and the reasoning — so that a reader can see *why* the project looks the way it does, not just what it looks like.

3.1 Emulation platform: QEMU ARMv8 "virt", not a Raspberry Pi 4 emulator

Decision

Use the QEMU "virt" Armv8-A reference board (the `qemu_v8` manifest in OP-TEE's own build tooling) as a stand-in for the Raspberry Pi 4, rather than looking for a Pi-4-specific emulator.

The original plan referred loosely to a "Raspberry Pi emulator". This needed to be made precise before any build work started, and the precise answer is less convenient than the phrase suggests: **there is no mainstream, officially maintained emulator that boots a Raspberry Pi 4 with working TrustZone/OP-TEE support.** Broadcom has never published the secure-boot and TrustZone configuration details for the BCM2711 SoC used in the Pi 4, so OP-TEE has no official port to that board, physical or emulated.

The platform actually used instead is a generic ARMv8-A machine with a proper TrustZone-capable CPU model and full, officially maintained OP-TEE support. This is the standard substitute used throughout the OP-TEE research and tooling ecosystem, and it shares the same CPU architecture family (ARMv8-A / AArch64) and the same secure/normal-world security model as the Pi 4's Cortex-A72. The target deployment platform remains the Raspberry Pi 4; QEMU is used to demonstrate the feasibility of the confidential-inference *design*, not to reproduce Pi 4 *performance*.

Honest cost of this substitution

QEMU/OP-TEE is a **functional** proxy, not a **performance** proxy. Two compounding factors make QEMU timings unrepresentative of a real Pi 4: (1) the `virt` platform does not model the Pi 4's actual Cortex-A72 cores, cache hierarchy, or memory bandwidth; and (2) because the OP-TEE build runs inside an x86_64 VirtualBox VM, QEMU performs full software instruction translation (TCG) from ARM64 to x86_64 with no KVM acceleration — and CPU-bound workloads such as neural-network matrix multiplications are exactly

what TCG handles worst. TCG-translated, CPU-bound workloads are commonly 5–20× slower than native execution; a real Pi 4 running natively should be expected to beat the QEMU/TCG setup by roughly one to two orders of magnitude for RawTFNet-16 inference specifically. This is recorded explicitly so that no latency number produced on QEMU is ever mistaken for a Pi 4 figure in the final thesis.

3.2 Development host: VirtualBox Ubuntu 22.04 LTS, not WSL2

Decision

Develop inside a VirtualBox virtual machine running Ubuntu 22.04 LTS with a **bridged** network adapter, instead of using WSL2 on the same Windows machine.

OP-TEE's build system (the `repo` multi-repository tool, Buildroot, device-tree compilation, cross-toolchains) requires a native Linux environment and does not build reliably under Windows compatibility layers such as Git Bash/MINGW64. A Linux environment was therefore required either way; the choice was between WSL2 and a full VM.

A VM was chosen over WSL2 specifically because a later stage of this work needs to receive live audio from an Asterisk server, and Asterisk's SIP/RTP traffic needs a stable, directly reachable network address. WSL2's default NAT layer and internal IP volatility are a known source of unreliability for real-time media traffic, whereas a VirtualBox VM with a bridged network adapter gives the Linux environment a stable IP on the same LAN as any calling client — while still letting Windows (model training, Python tooling) and Linux (OP-TEE, Asterisk) run side by side without dual-boot reboots. Ubuntu 22.04 LTS specifically was chosen for its long-term support and strong compatibility with VoIP frameworks, security libraries, and research tooling, reducing the risk of dependency conflicts through the rest of the project.

In practice the VM's bridged-network IP address changed across several reboots during the project (observed values included `192.168.1.183`, `192.168.1.175`, and `192.168.0.11`), which became a minor recurring friction point — addressed simply by re-checking `ip a` before each file transfer rather than assuming a fixed address.

3.3 Model deployment path: PyTorch → ONNX → C, not hand-ported C

Decision

Export the trained PyTorch model to ONNX, verify it numerically at every stage, and compile the verified graph directly to standalone C for embedding in a Trusted Application — rather than hand-porting RawTFNet's layers to C by hand.

A Trusted Application executes inside a minimal, single-threaded environment with a restricted C library and a small, fixed secure-memory heap; it cannot host a full deep-learning framework. PyTorch, ONNX Runtime, and TensorFlow all assume a POSIX-ish environment with dynamic memory, threads, and large heaps, none of which exists inside a TA. Two options were considered:

- **Hand-port RawTFNet's layers directly to C.** Rejected as high-effort and high-risk: every convolution, the Sinc layer, the squeeze-and-excitation modules, and the batch-norm folding would need to be reimplemented and independently verified, with numerous opportunities for silent numerical divergence from the trained model.

- **Export to ONNX (a framework-neutral graph representation), then compile it ahead-of-time to dependency-free C.** Adopted. Correctness is checked numerically at every conversion step against the original PyTorch output, so that any discrepancy is caught at the specific stage that introduced it, rather than surfacing later inside the TEE where debugging is significantly harder.

Within this second path, two different tools were tried for the ONNX-to-C step, and one of them was abandoned. This is documented in detail in Section 5.3, because it was a real dead end that cost real time, not a clean first-try success.

3.4 Apache TVM: used for independent verification, not for the deployed code

Apache TVM (0.25.0.post1) was used at one stage of the pipeline, but strictly as an **independent cross-check** of the ONNX graph's correctness — compiling it with TVM's Relax API and an LLVM JIT target, and confirming the output matched PyTorch and ONNX Runtime to four decimal places on a held-out test utterance. TVM was deliberately *not* used to generate the C code that ships inside the deployed Trusted Application; that role belongs to the tool described in Section 5.3 (`emx-onnx-cgen`). Keeping these two roles separate meant that a numerical mismatch could always be traced to either the export step or the final C codegen step, never to an ambiguous mixture of both.

One practical difficulty is worth recording for reproducibility: the installed TVM release (0.25.0.post1) uses the newer Relax API, which has fully replaced the classical Relay API that most published TVM tutorials and documentation still assume. Import paths such as `tvm.relay` and `tvm.nd` no longer exist; the working incantations (`tvm.relax.frontend.onnx.from_onnx`, `tvm.runtime.tensor`) had to be found empirically by inspecting the installed package rather than by following an existing guide.

3.5 Scope of the proof-of-concept

The full thesis scope combines RawTFNet, TrustZone confidential inference, and federated learning with differential privacy. That full scope was deliberately narrowed for this phase of the work to demonstrating the confidential-inference path alone: export the already-trained model and get it executing inside the TrustZone/OP-TEE environment, deferring federated learning, weight re-training, and secure key-provisioning for weights to later work. The model's weights are compiled directly into the TA binary as a static array for this PoC; this is a known simplification — it means the TA would need re-signing on every retrain — and is recorded explicitly in the limitations (Chapter 7).

Chapter 4

System Architecture

Figure 4.1 shows the target end-to-end pipeline: from a live call, through a rolling audio buffer and windowing in the normal world, across the TrustZone boundary into the secure-world classifier, and back out to a live decision. The colour of each stage reflects how this document is organised: green stages are the model-correctness work (Chapter 5 onward covers how they were achieved), and everything to the right of the dashed confidential-inference boundary is protected from a compromised normal-world host.

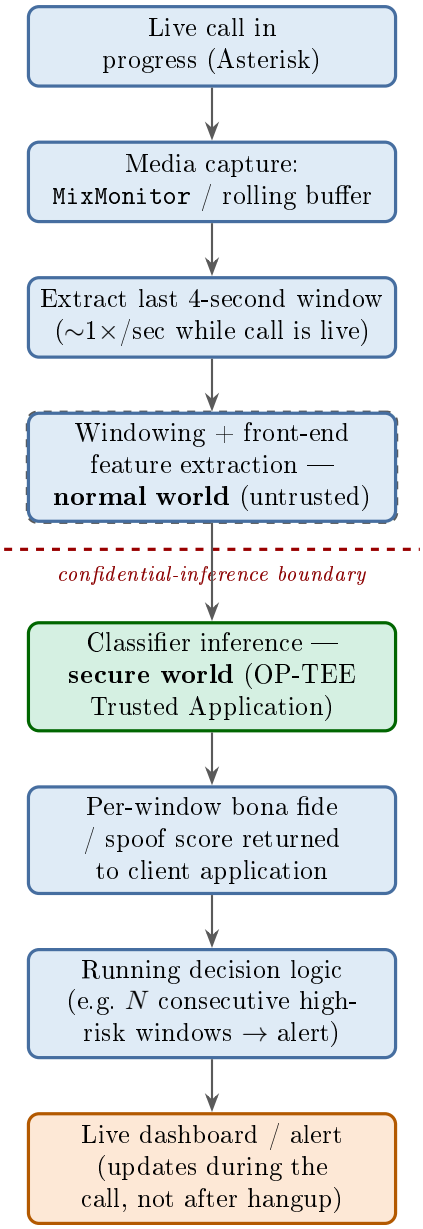


Figure 4.1: End-to-end live in-call detection pipeline. Feature extraction (the front-end half of RawTFNet) currently remains in the untrusted normal world; only the classifier's forward pass executes inside the secure world for this PoC (Section 5.5 explains why the model is split at exactly this point). This is a deliberate, stated scope boundary, recorded again in Chapter 7.

Chapter 5

The Implementation Journey

This chapter is the core of the document: a chronological account of every phase of the work, including the phases that failed on the first attempt, the phases that failed on the *second* attempt for a different reason, and the one significant misdiagnosis that was caught and corrected before it could mislead later work.

5.1 Phase 1 — Environment bring-up

The purpose of this stage was to confirm that the toolchain, cross-compilation, and QEMU boot process all worked before any project-specific code was written. OP-TEE OS, TF-A, the Linux kernel, and a Buildroot root filesystem were built for the QEMU Armv8-A `virt` platform from the official manifest, and the environment was booted and the official OP-TEE test suite run against it.

Result

System booted with a working secure-world / normal-world split. `xtest` completed with **0 failures across 41,482 subtests and 145 test cases**. This is the standard signal that the secure/normal-world split is functioning correctly on this platform.

Several ordinary build-dependency errors were hit and fixed along the way and are recorded here for completeness rather than as anything remarkable: `libgnutls28-dev` was missing (u-boot's `mkeficapsule` build failed until it was installed); `python3-tomli` was missing (the QEMU meson build failed until it was installed); and one transient HTTP 404 on a QEMU subproject download resolved itself on retry.

5.2 Phase 2 — Validating the TA/CA communication pattern

Before writing any model-specific code, the official `hello_world` TA/CA example was built and run to validate the exact client-application-to-trusted-application invocation pattern the model-inference TA would later reuse.

Result

The client application successfully opened a session with the trusted application and received a correctly computed result back (the TA incremented a test value from 42 to 43). This de-risks the communication pattern itself, so that Milestones 4 onward could be scoped as "plug the verified model into a known-working pattern" rather than an open integration risk.

5.3 Phase 3 — Exporting and compiling the model

5.3.1 Export to ONNX and numerical verification

The trained checkpoint was exported to ONNX with a fixed input shape (no dynamic axes) and all weights consolidated into a single file rather than external-data references, and its output was verified against the original PyTorch model on a held-out ASVspoof 2021 LA utterance. Both produced the same score to four decimal places (PyTorch −5.6216, ONNX −5.6216; TVM/Relax cross-check −5.621588).

One export-time error surfaced during this stage and is worth recording because it reveals a structural property of the model, not a tooling bug: attempting to export or run the model on an input shorter than the native 64,600-sample window fails with

```
RuntimeError: Given input size: (24x5x1). Calculated output size:
(24x2x0). Output size is too small
```

because the network's internal max-pooling layers collapse a too-short input to zero spatial size. This confirmed, independently of the later memory-wall investigation, that the 4-second window is not an arbitrary convenience but a structural requirement of the trained architecture — a point the memory-wall investigation in Section 5.4 returns to directly.

5.3.2 The abandoned path: onnx2c

`onnx2c` was the first ONNX-to-C tool tried, and it was abandoned after three consecutive operator-support gaps made it unusable for this model.

`onnx2c` was cloned and built (using the targeted `make onnx2c` build, since the tool's own full test suite fails on an unrelated `_Float16` target issue that has nothing to do with this model). Three separate blocking errors were then hit in sequence while trying to compile the cleaned ONNX graph:

1. An external-data tensor reference the tool could not resolve (fixed independently by re-exporting ONNX with all weights embedded in one file).
2. `Unknown attribute training_mode` on `BatchNormalization` nodes — partially worked around with `onnxsim` simplification and a script that manually stripped the attribute, but this treated a symptom rather than a root cause.
3. `No axes to squeeze away?` on a `Squeeze` node inside the squeeze-and-excitation (SE) module — this one could not be resolved. The SE module's data-dependent squeeze pattern fell outside what `onnx2c`'s parser supported, and no workaround was found.

At this point `onnx2c` was abandoned for this model, and `emx-onnx-cgen` was adopted instead.

5.3.3 The adopted path: emx-onnx-cgen

`emx-onnx-cgen` was chosen over continuing to patch `onnx2c` for concrete reasons: it is pip-installable, reports over 99% coverage of the ONNX opset-26 operator set, generates static, malloc-free-by-design[1] C, and ships a built-in `verify` command that checks the generated C against ONNX Runtime with explicit ULP (unit-in-the-last-place) tolerance reporting.

[1] As Section 5.4 shows, "malloc-free by design" described the tool's default posture, not this project's eventual configuration — the setting that maximises static allocation (`-large-weight-threshold 0`) was exactly the one that later caused the load failure. This is flagged here and resolved fully in Section 5.4.

The classifier half of the model (the half eventually embedded in the TA; see Section 5.5) compiled cleanly and its `verify` command reported a maximum deviation of **7 ULP** against ONNX Runtime — an essentially exact match, several orders of magnitude tighter than would be needed to change a bona-fide/spoof decision.

5.4 Phase 4 — The memory wall

This is the most significant obstacle encountered in the entire confidential-inference PoC, and the one that took the longest to correctly diagnose. It is presented here as a full investigation narrative, in the order the reasoning actually unfolded, including the theories that turned out to be wrong.

5.4.1 The failure

With the full model (front-end and classifier together) compiled to a single C file and linked into a Trusted Application, the very first attempt to open a session against it failed. The client application reported

```
TEEC_OpenSession failed 0xffff000c
```

and the secure-world log showed the load itself failing before the TA ever ran a single instruction:

```
E/LD:  populate_segments:891 sys_map_zi
E/TC:?  1 ldelf_init_with_ldelf:152 ldelf failed with res:  0xffff000c
D/TC:?  1 tee_ta_open_session:722 init session failed 0xffff000c
```

`sys_map_zi` is the loader step that maps a TA's zero-initialised memory (its `.bss` segment and heap reservation) into the secure world before execution starts. A failure here means the TA's declared memory footprint could not be satisfied by the loader — not a bug in the model's logic, but a resource ceiling.

5.4.2 Investigation timeline

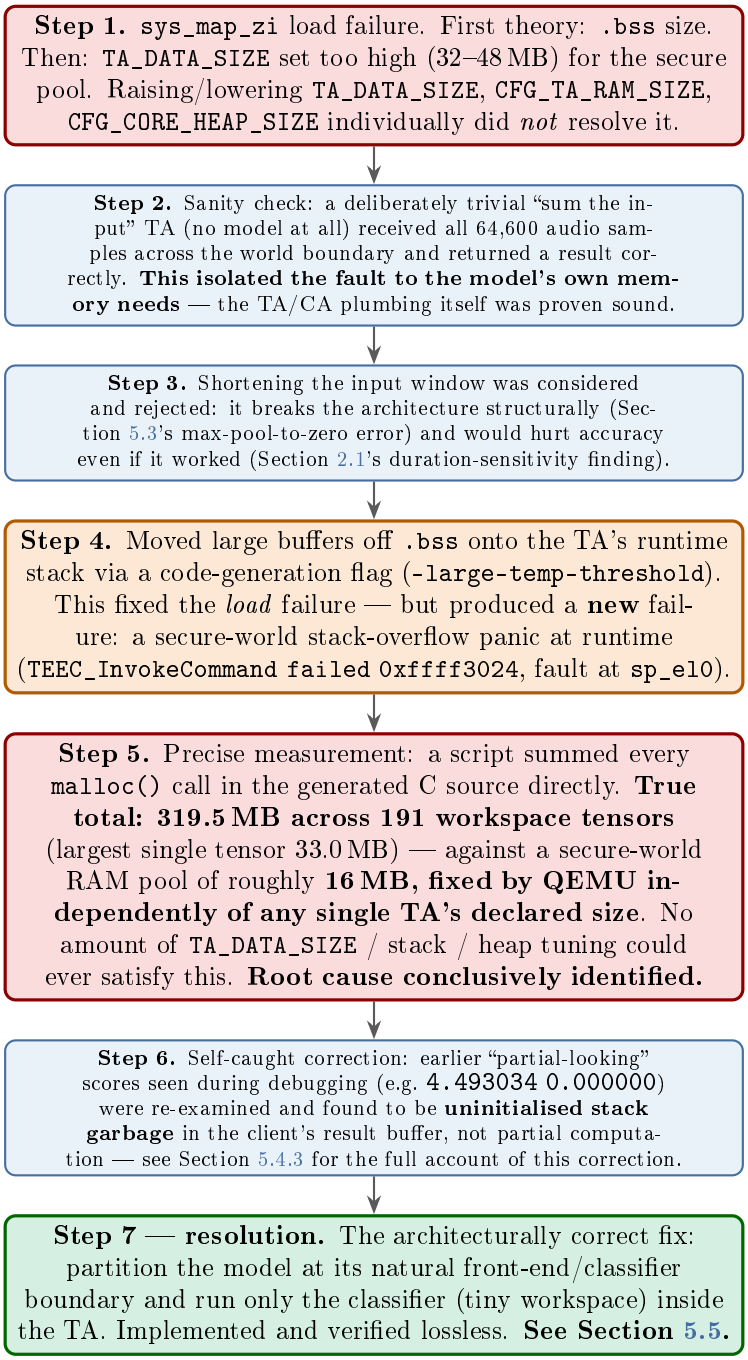


Figure 5.1: Chronology of the memory-wall investigation, in the order it actually happened. Each step either eliminated a hypothesis or narrowed the search, and the final measurement in Step 5 is what turned an open-ended debugging problem into a conclusively settled root cause.

5.4.3 The self-caught misdiagnosis

During Step 4's debugging, the TA occasionally returned scores such as `4.493034 0.000000` or `6.403191 0.000000` instead of either a clean result or an outright failure. The first working theory was that these represented *partial computation* — as if the model were somehow tracking how far it got before running out of heap. **This theory was wrong**, and it was caught and corrected before it could shape the eventual fix.

Reading the generated C source directly revealed the actual mechanism: `emx-onnx-cgen`'s generated `model()` function allocates its ∼191 workspace tensors via a long sequence of individual `malloc()` calls, and if *any* of those calls fails, the function returns silently **without writing to the output buffer at all**. The values that looked like "partial progress" were simply whatever bytes happened to already be sitting in the client's uninitialised `logits` array on the stack — an artifact of C's lack of default zero-initialisation, not a signal about the model. Once this was understood, the TA handler was changed to zero-initialise `logits[2] = {0.0f, 0.0f}` before calling into the model, so that any future silent allocation failure would show up as an unambiguous exact-zero result rather than a plausible-looking but meaningless number. This fix is small, but the diagnostic discipline behind it — reading the generated source rather than trusting a superficially plausible pattern in the output — is what prevented a wrong root cause from being chased further.

5.5 Phase 5 — The fix: partitioning the model

Resolution

Rather than trying to fit a 319.5 MB workspace into a ∼16 MB secure-memory pool, the model was split at its natural architectural seam — the boundary between the front-end (feature-extraction) stage and the classifier stage already visible in Figure 2.1 — and only the small classifier half was compiled into the Trusted Application.

This works because RawTFNet's own `forward()` method is already exactly `classifier(front_end(x))`: the split point already exists in the trained model's own structure, so no retraining or architecture surgery was needed, only a different export boundary.

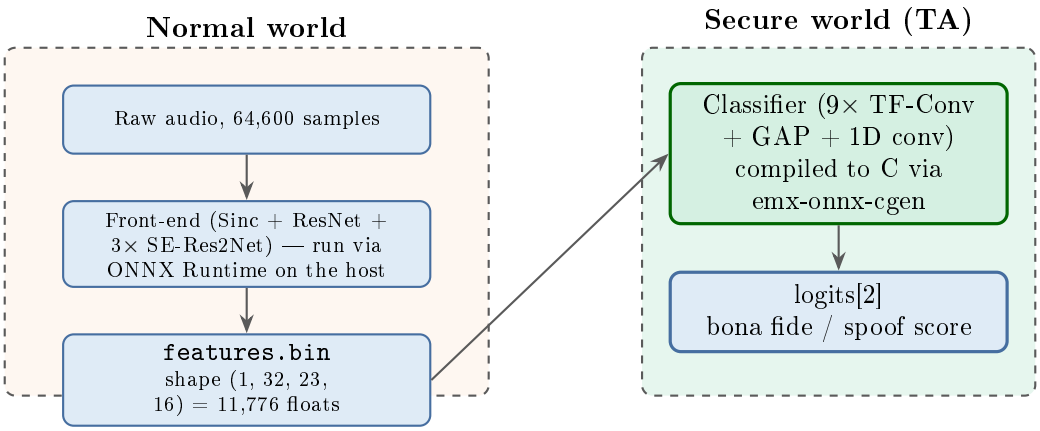


Figure 5.2: The partitioned deployment. The front-end runs unmodified in the normal world (feature extraction is explicitly *not* part of the confidentiality boundary for this PoC — recorded as a stated limitation in Chapter 7); only the classifier's forward pass, with its far smaller workspace, executes inside the secure-world Trusted Application.

5.5.1 Verifying the split is lossless

Splitting a working model always raises one question: does the split itself introduce any error? This was checked directly rather than assumed. `model.front_end(x)` was run in PyTorch to produce reference features, saved to `features.bin`; `model.classifier(features)` was run separately and compared against the full model's single forward pass on the same input.

Verification result

Classifier-only output matched the full-model forward pass **exactly**: `[[5.391142 −5.6215887]]` in both cases. The classifier's exported ONNX graph, compiled through `emx-onnx-cgen`, then matched this reference at **7 ULP** maximum deviation. Final TA configuration: `TA_DATA_SIZE` 4 MB, `TA_STACK_SIZE` 256 KB — comfortably inside the secure pool that the full model could never fit into. The TA loaded, ran, and returned `logits: 5.391142 -5.621588` from inside the secure world, matching the PyTorch reference to six decimal places.

The pipeline was then verified on a *second*, independent audio file (`LA_E_1001928.flac`) to confirm this was not a one-off coincidence: PyTorch scored it −4.9850; the secure-world TA scored the same file −4.985014 — confirming the pipeline generalises across inputs, not just the one file used during development.

5.6 Phase 6 — Any-audio pipeline and proof of secure-world execution

With the partitioned model working, the pipeline was generalised to accept any user-supplied audio file rather than a single fixed test vector, and then hardened for demonstration purposes.

5.6.1 The any-audio pipeline

A host-side script (`prepare_audio.py`) loads an arbitrary `.wav`/`.flac` file via `librosa` (which resamples to 16 kHz automatically, resolving any sample-rate mismatch), peak-normalises it, pads or repeats it to 64,600 samples, runs it through the exported front-end ONNX graph, and writes the resulting features to a fixed shared-folder location that the QEMU guest reads over a 9p virtio mount. The guest-side client application was changed to read this fixed path automatically, so that a full round trip requires only `optee_example_rawtfnet` on the guest side with no arguments, once the host-side preparation script has been run.

5.6.2 Proving *which* audio was processed, and that it happened in the secure world

Requirement raised during review

It is not enough to show a correct-looking score; a reviewer needs independent evidence that (a) the computation genuinely happened inside the secure world, and (b) it happened on the specific audio file being claimed, not a stale or substituted result.

This was addressed by adding a second TA input parameter carrying the original filename as a label, and logging it from *inside* the secure world via `IMSG`: `=== SECURE WORLD: processing audio file: <name> ===`, followed by a second `IMSG` line after inference with the computed logits. Because this line is written to the Secure World console — a log stream the normal-world client application has no way to fabricate or edit — it functions as independent, tamper-resistant evidence of exactly which file was processed and that the processing occurred inside TrustZone.

One build-configuration detail nearly hid this evidence entirely and is worth recording: `ta/Makefile` sets `CFG_TEE_TA_LOG_LEVEL`, which defaults to `1` in the copied example project. At that level, info-level `IMSG` output — including both proof lines — is filtered out of the console, so the TA appeared to be running correctly while producing no visible evidence at all. Raising the level to `3` made the proof lines appear as intended. This was first discovered when a build that had aborted partway through (on the `cryptography`-module error described below) left QEMU booting a *stale* root filesystem image on the next run, so the new logging code appeared not to be running even though the underlying TA logic was already correct — two separate causes stacked on top of each other, both resolved by (1) ensuring the full `make` completes without error before rebooting, and (2) raising the log level.

5.7 Phase 7 — The domain-generalisation investigation

Real-world audio — Waleed's own recorded voice, an ElevenLabs text-to-speech spoof, and audio supplied independently by the supervisor — consistently misclassified as **SPOOF** through the pipeline, including genuine bona fide recordings. The pipeline itself was working correctly (verified repeatedly in Phases 5–6); this is a property of the *model*, investigated as a research question in its own right rather than treated as a bug to silently patch around.

This investigation proceeded as a controlled elimination study: each plausible cause was tested and either fixed or ruled out, one at a time, before moving to the next.

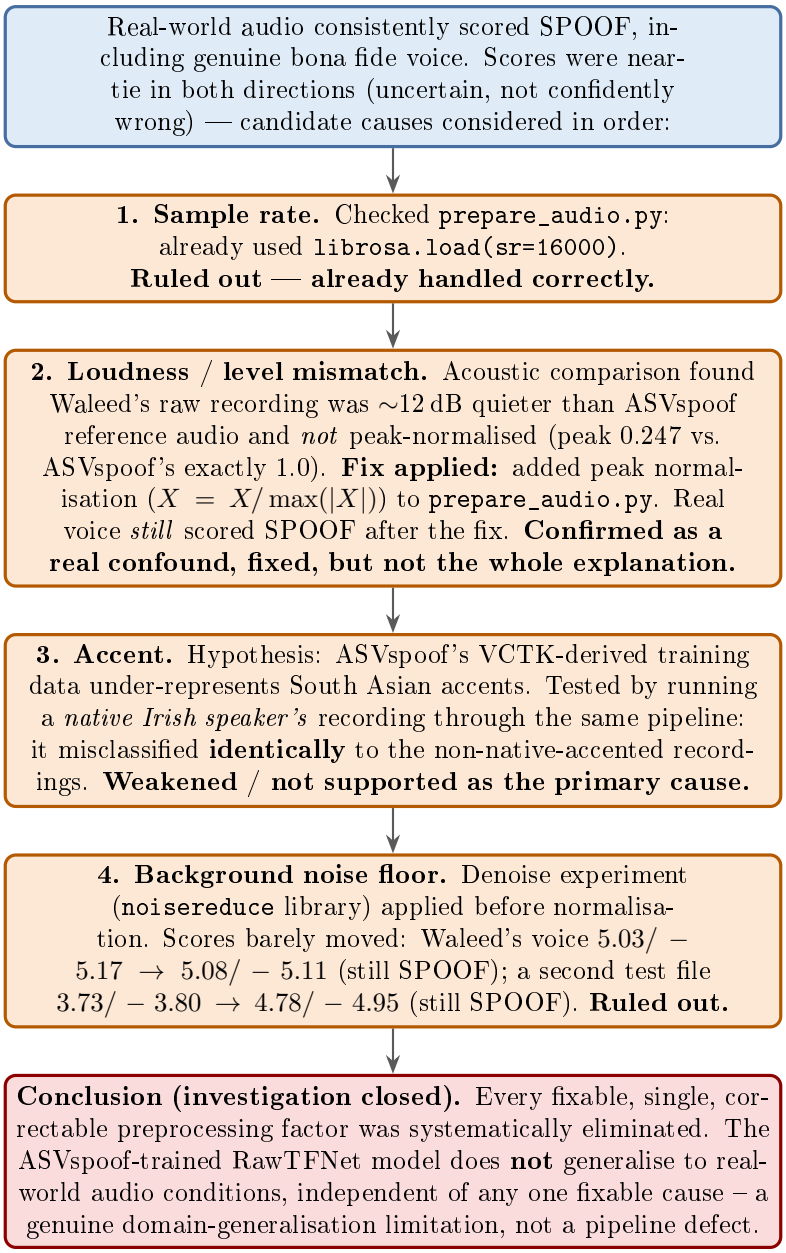


Figure 5.3: The out-of-distribution elimination study: each candidate cause of real-world misclassification was tested individually before being ruled in, ruled out, or fixed. Sample rate was already correct; loudness was a real, fixable confound that was fixed but did not resolve the misclassification; accent was weakened as an explanation by a same-pipeline native-speaker control; noise floor was ruled out directly. What remains is attributed to the trained model keying on dataset-specific synthesis artefacts of ASVspoof rather than general spoof characteristics.

A refinement, not a reversal

Audio clips later supplied independently by the supervisor (Dr. Foley) as an out-of-sample speaker test refined rather than overturned this conclusion. One clip (`Rec 1.flac`) scored bona fide 0.54 vs. spoof −0.74 — still classified SPOOF (incorrect, it was a genuine recording), but with scores an order of magnitude closer to the decision boundary than every prior real-world file, which had scored confidently in the 3–5 range. This suggests the domain gap is better described as a **spectrum** than a wall: the degree of misclassification varies by recording, and this particular clip landed almost exactly on the fence. The remaining four

supplied clips (`Rec 2-5.flac`) are queued to run through the same pipeline as a next step.

5.7.1 Why this is framed as a finding, not a failure

Framed correctly, this investigation is legitimate thesis content, not a setback to hide. The confidential-inference gateway itself — the actual subject of this proof-of-concept — works flawlessly, proven end-to-end with independent secure-world evidence (Phase 6). The detector's real-world accuracy is a separate, now rigorously characterised research problem, with a clear recommended path forward: report batch accuracy/EER on in-distribution ASVspoof evaluation data, present this elimination study as a results/limitations chapter, and name domain adaptation or fine-tuning on real-world and TTS audio as explicit future work. This mirrors exactly the generalisation gap that the project's own literature review identifies as an open problem across the field (Chapter 8, RQ1), which places this project's own empirical finding in direct conversation with the surrounding literature rather than as an isolated anecdote.

5.8 Phase 8 — Live Asterisk call integration

The final phase of the work in progress at the time of writing is wiring the verified secure-world pipeline to a live phone call, so that a verdict streams during the call rather than after it ends.

5.8.1 Design: sliding window, not record-then-analyse

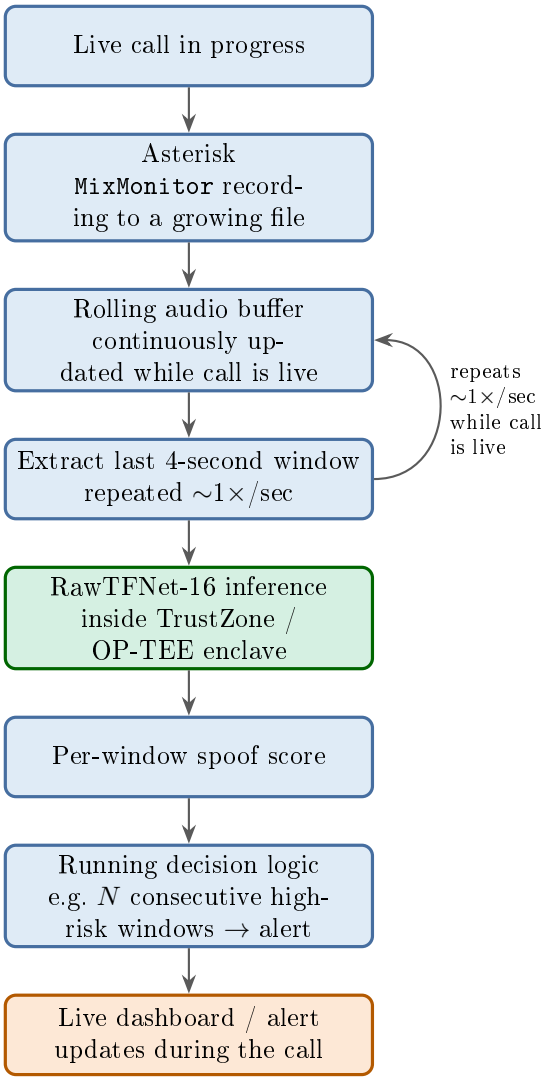


Figure 5.4: Live in-call sliding-window design. A single noisy window is prevented from triggering a false alarm mid-call by requiring several consecutive high-risk windows before an alert is raised, the same overlapping-chunk pattern used by live captioning or live translation systems.

The 4-second window length is not arbitrary: it matches RawTFNet's own training segment length and the duration-sensitivity finding recorded in Section 2.1 (accuracy is worst on clips under 2 seconds). Shortening the window to reduce latency was considered and rejected for exactly this reason.

5.8.2 What this requires from Asterisk, and what was actually built for this PoC

A genuinely live pipeline would stream audio out of Asterisk while the call is still connected (via AudioSocket or the Asterisk REST Interface with external media) into a long-running detection service. For this proof-of-concept phase, a pragmatic intermediate step was built instead: Asterisk was installed on the Ubuntu VM with a SIP endpoint and a dialplan extension that answers a call and records it continuously via `MixMonitor` to a growing file; a host-side watcher script (`live_detect.py`) polls that growing recording, and every 4 seconds extracts the newest 64,600-sample window, peak-normalises it, runs it through the front-end, and signals the QEMU guest

to run a fresh secure-world inference — reproducing the sliding-window behaviour of Figure 5.4 against a call that is still being recorded, ahead of migrating to a true streaming capture method.

Status at time of writing

Asterisk installation, SIP endpoint (`100`) and dialplan (extension `600`) configuration, and MicroSIP softphone registration are complete and confirmed working: a test call was placed and the recording file (`/tmp/live.wav16`) was observed growing in real time during the call. The full end-to-end demonstration — watcher loop plus `live_detect.py` plus a live call producing a stream of secure-world verdicts — has been re-explained in full and is the next concrete milestone, not yet reported as executed end-to-end at the time of writing.

Chapter 6

Verification Summary

Table 6.1 consolidates every point in the pipeline where a numerical or behavioural claim was independently checked, rather than assumed, because this is the evidence base the thesis's correctness claims ultimately rest on.

Checkpoint	Evidence
OP-TEE environment	`xtest`: 0 failures across 41,482 subtests, 145 test cases
TA/CA communication pattern	`hello_world` round trip: value 42 correctly incremented to 43 across the secure-world boundary
PyTorch → ONNX export	Scores match to four decimal places on a held-out ASVspoof utterance (PyTorch −5.6216, ONNX −5.6216)
ONNX → TVM (independent cross-check)	TVM/Relax output −5.621588, matching both prior stages
ONNX → C (emx-onnx-cgen)	Built-in `verify` against ONNX Runtime: max deviation 7 ULP
Front-end / classifier split	Classifier-only output identical to full-model forward pass: `5.391142`, `−5.6215887` in both cases
In-TA classifier execution	Secure-world TA returned `5.391142 -5.621588`, matching the PyTorch reference to six decimal places
Generalisation to a second file	PyTorch −4.9850 vs. secure-world −4.985014 on an independent test file (`LA_E_1001928.flac`)
Secure-world execution proof	`IMSG` filename + score logged from inside the secure world, independently visible on a console the normal-world CA cannot write to

Table 6.1: Every correctness or behavioural claim in this document that was independently verified, with the evidence produced at each stage.

Chapter 7

Limitations and Threats to Validity

These are stated explicitly here, in the same form intended for the thesis's own Limitations section, so that nothing in this document overstates what has actually been demonstrated.

Limitation	Explanation
No physical target hardware	All results were obtained on the QEMU ARMv8-A `virt` reference platform, not a physical Raspberry Pi 4. This validates the software architecture and the TrustZone isolation model, not device-specific behaviour (boot chain, peripheral timing, thermal characteristics of the BCM2711 SoC).
No hardware root of trust	QEMU's secure world is a software CPU-mode partition: no fused keys, no physical secure boot chain, no tamper resistance. Hardware-backed attestation is out of scope and identified as future work.
Static, compiled-in model weights	The PoC keeps model weights as a static array compiled directly into the TA binary, skipping secure storage / key-provisioning. Not workable for the longer-term federated-learning goal, but a reasonable simplification for a feasibility PoC.
Feature extraction stays in the normal world	Only the classifier's forward pass runs inside the secure world; audio windowing and front-end feature extraction remain in the untrusted normal world. This is a deliberate, stated scope boundary, not an oversight — an attacker with normal-world root could in principle feed the TA manipulated features.
Emulation performance is not representative	QEMU/TCG software translation is expected to be one to two orders of magnitude slower than native Pi 4 execution for this workload. Any latency claims from this phase must be reported explicitly as QEMU-only figures.
Real-world detection accuracy	Systematically characterised in Section 5.7: the ASVspoof-trained model does not reliably generalise to real-world recording conditions. This is reported as a research finding with a defined future-work path (domain adaptation / fine-tuning), not hidden or worked around.
Live capture is not yet streaming	The Asterisk integration currently polls a growing `MixMonitor` recording rather than receiving a true live audio stream (AudioSocket / ARI external media); this is the next milestone.
Toolchain instability	Apache TVM 0.25.0.post1's Relax API differs enough from most published tutorials that import paths had to be resolved empirically; pinning a specific, documented TVM version is recommended before final submission.
Timing / side-channel hardening out of scope	No side-channel hardening (cache-timing, power analysis) has been attempted. Reasonable to state explicitly as future work rather than an oversight.

Table 7.1: Consolidated limitations and threats to validity.

Chapter 8

Positioning Against the Literature Review

The project's literature review is organised around three research questions; this proof-of-concept speaks most directly to the second, and its own empirical findings connect back to the first.

RQ	How this PoC relates
RQ1	Which post-2020 detection architectures best balance EER against edge computational footprint? RawTFNet-16 was selected precisely on this basis (Table 2.1). This project's own out-of-distribution elimination study (Section 5.7) is direct, first-hand evidence for the review's finding that accuracy drops sharply once audio moves from clean academic datasets to real-world conditions.
RQ2	How can federated learning, differential privacy, and TrustZone-based confidential inference be combined for end-to-end privacy guarantees? No peer-reviewed study combines all three specifically for anti-spoofing; the literature review treats their combination as a proposed synthesis rather than something already validated. **This PoC is the first concrete, working piece of that synthesis** — it demonstrates that the TrustZone/confidential-inference primitive alone is achievable end-to-end with a real anti-spoofing model, which is the load-bearing precondition for layering federated learning and differential privacy on top in subsequent work.
RQ3	What deployment challenges remain in moving detection from lab benchmarks to the residential telephony edge? The Asterisk integration work (Section 5.8) is a direct, concrete attempt at exactly this transfer, and the domain-generalisation finding is first-hand evidence of the widening lab-to-real-world performance gap the review identifies as a field-wide open problem.

Table 8.1: How this proof-of-concept's work maps onto the thesis's three research questions.

Chapter 9

Current Status and Next Steps

9.1 Where the project stands

- The confidential-inference core is **complete and verified end-to-end**: a partitioned RawTFNet-16 classifier runs inside an OP-TEE Trusted Application on QEMU, produces scores matching the original PyTorch model to six decimal places, and provides independent secure-world evidence of which audio file it processed.
- The any-audio pipeline is working and has been demonstrated on multiple files, including ASVspoof evaluation clips (correct classification) and real-world recordings (the characterised domain-generalisation limitation of Section 5.7).
- Asterisk is installed, configured, and confirmed capturing live call audio; the watcher-loop integration with the secure-world pipeline has been designed and documented but not yet reported as run end-to-end.

9.2 Immediate next steps

1. Run the remaining supervisor-supplied clips (`Rec 2-5.flac`) through the pipeline to extend the near-boundary observation from `Rec 1.flac` into a small independent-speaker result set.
2. Complete and report the first full end-to-end live-call demonstration: watcher loop, `live_detect.py`, and a live call producing a visible stream of secure-world verdicts.
3. Produce a batch accuracy / EER measurement across a larger sample of ASVspoof evaluation files (20–50 files), to report a proper in-distribution accuracy figure alongside the out-of-distribution findings.
4. Add TA-side timing instrumentation to capture clean QEMU inference-latency numbers, explicitly labelled as non-representative of Pi 4 performance per Section 3.1, as a baseline for later comparison if physical Pi 4 hardware becomes available.
5. Migrate the live-call capture path from polling a growing `MixMonitor` recording to a genuine streaming source (AudioSocket or ARI external media), closing the last gap identified in Table 7.1.

Chapter 10

Summary

The confidential-inference phase of this thesis set out to answer one question: can a trained audio-deepfake detector be made to run inside an ARM TrustZone secure world, with its correctness provable end-to-end, on hardware actually available during this phase of the project? The answer, demonstrated and verified repeatedly rather than assumed, is yes — but the path there was not straightforward, and this document has tried to be honest about that rather than presenting only the final working state.

Three points are worth restating together, because they are the ones a supervisor is most likely to probe: first, every substitution made along the way — QEMU for a physical Pi 4, a VM for WSL2, ONNX compilation for hand-ported C — was a deliberate, justified engineering decision with a stated cost, not an unexamined convenience. Second, the memory-wall failure in Section 5.4 was a genuine, multi-day obstacle that was resolved by measurement and architectural redesign (partitioning the model at its natural seam) rather than by parameter tuning, and the self-caught misdiagnosis along the way is included deliberately, as evidence of the diagnostic process rather than something to omit. Third, the real-world misclassification finding in Section 5.7 was investigated with the same rigour as any other result in this document — systematically, one variable at a time — and the conclusion that emerged is a legitimate research contribution about the limits of the ASVspoof-trained model, directly connected to a gap the thesis's own literature review already identifies as open in the field.

The confidential-inference gateway — the actual subject of this proof-of-concept — works. What remains is extending it to a fully streaming live-call demonstration, and reporting the detector's accuracy honestly across both the conditions it was trained for and the conditions it was not.